

Relatório de Deploy

Lançamento Oficial do MVP do Aplicativo MenteUni

Projeto	MenteUni – Aplicativo de Saúde Mental para Estudantes
Disciplina	Projeto Mobile
Estratégia	Opção B: Distribuição Controlada (Firebase App Distribution)
Versão	1.0 (versionCode 1)
Data	11 de novembro de 2026

Equipe

Igor Moreira de Sousa	RA 0022246
João Marques de Araújo Neto	RA 0022809
Wenderson da Silva Souza	RA 0022897
Victor Garcia Peres de Melo	RA 0027039

Centro Universitário do Planalto Central Apparecido dos Santos (UNICEPLAC)
Brasília – DF, 2026

1. Entrega

Item	Valor
Link de Acesso (convite Firebase App Distribution)	**https://appdistribution.firebase.dev/i/7d180195a46b49c6**
API em produção	https://menteuni-api.onrender.com
Documentação da API (Swagger)	https://menteuni-api.onrender.com/swagger-ui.html
Código-fonte	https://github.com/Igorms27/Menteuni
Pacote Android	br.edu.uniceplac.menteuni
Artefatos gerados	MenteUni-v1.0.apk (3,5 MB, distribuição) e MenteUni-v1.0.aab (3,8 MB, lojas)

2. Visão geral e arquitetura

O MenteUni é um aplicativo para os estudantes da UNICEPLAC acompanharem a própria saúde mental: check-in emocional (humor, estresse e ansiedade), questionários de bem-estar, registro de sono, histórico com gráficos, dicas de saúde e alertas automáticos gerados pelas regras de negócio do back-end.

A arquitetura segue o documento definido no início do semestre: cliente-servidor em três camadas.

Camada	Tecnologias	Ambiente de produção
Aplicativo mobile	Ionic 8, Angular 20, TypeScript, Capacitor 7, Chart.js	APK Android assinado, distribuído pelo Firebase App Distribution
Back-end (API REST)	Java 21, Spring Boot 3.5, Spring Security (JWT), Spring Data JPA, Flyway, OpenAPI/Swagger, Docker	Render (Web Service, container Docker)
Banco de dados	MySQL 8	Aiven for MySQL (conexão com SSL obrigatório)

```
App (Ionic/Angular)  ->  API (Spring Boot, Render)  ->  MySQL (Aiven)
```

3. Configuração de variáveis de ambiente

3.1 Aplicativo: URL da API de produção

O Angular usa um arquivo de ambiente para desenvolvimento e outro para produção. No build de release o arquivo de produção substitui o de desenvolvimento automaticamente (`fileReplacements` no `angular.json`).

```
// src/environments/environment.ts  (desenvolvimento)
export const environment = { production: false, apiUrl: 'http://localhost:8080/api' };

// src/environments/environment.prod.ts  (produção)
export const environment = { production: true, apiUrl: 'https://menteuni-api.onrender.com/api' };
```

3.2 Back-end: credenciais por variáveis de ambiente

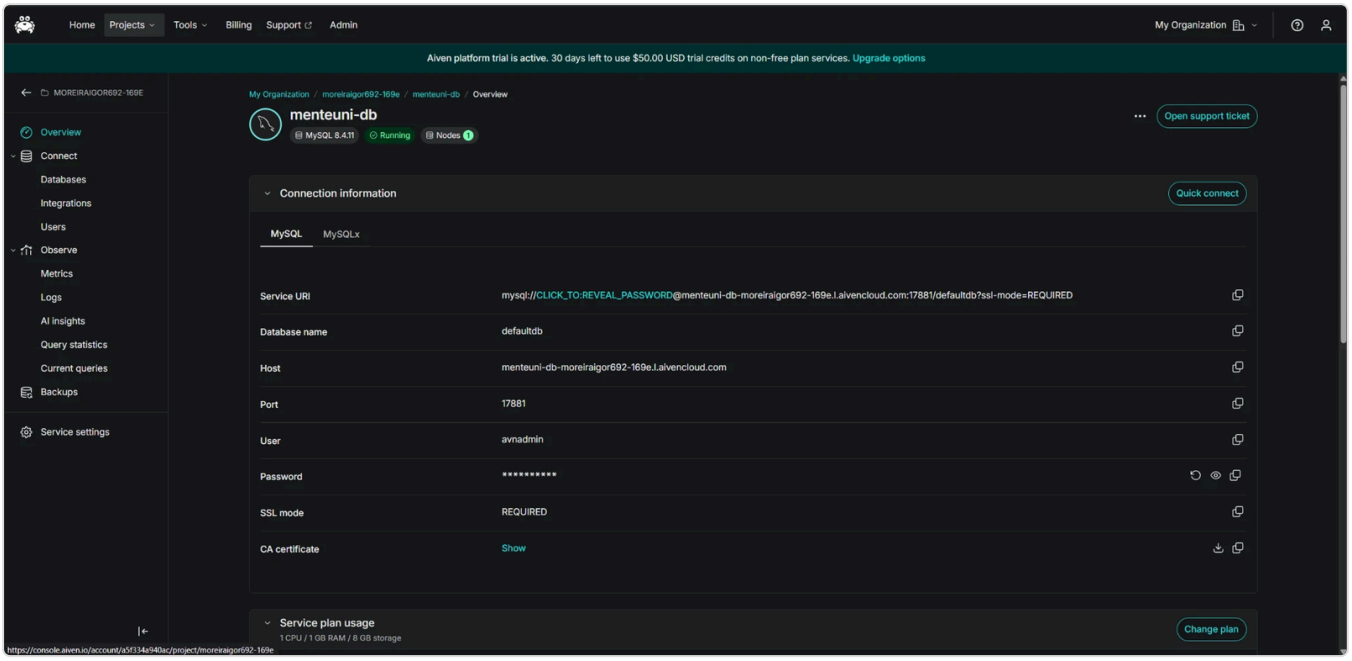
Nenhuma senha fica no código. O `application.yml` lê tudo de variáveis de ambiente, com valores padrão apenas para rodar localmente:

```
spring:
  datasource:
    url: ${DB_URL:jdbc:mysql://localhost:3306/menteuni...}
    username: ${DB_USER:root}
    password: ${DB_PASSWORD:root}
menteuni:
  jwt:
    secret: ${JWT_SECRET:...}
  cors:
    origens: ${CORS_ORIGINS:http://localhost:4200,https://localhost,capacitor://localhost}
```

No Render, o serviço foi criado a partir do arquivo `render.yaml` (Blueprint). As variáveis sensíveis (`DB_URL`, `DB_USER`, `DB_PASSWORD`) foram preenchidas no painel. O `JWT_SECRET` é gerado automaticamente pelo Render.

```
services:
  - type: web
    name: menteuni-api
    runtime: docker
    plan: free
    dockerContext: ./api
    dockerfilePath: ./api/Dockerfile
    healthCheckPath: /v3/api-docs
    envVars:
      - key: DB_URL        # jdbc:mysql://<host-aiven>:17881/defaultdb?sslMode=REQUIRED
        sync: false
      - key: DB_USER
        sync: false
      - key: DB_PASSWORD
        sync: false
      - key: JWT_SECRET
        generateValue: true
      - key: CORS_ORIGINS
        value: https://localhost,http://localhost,capacitor://localhost
```

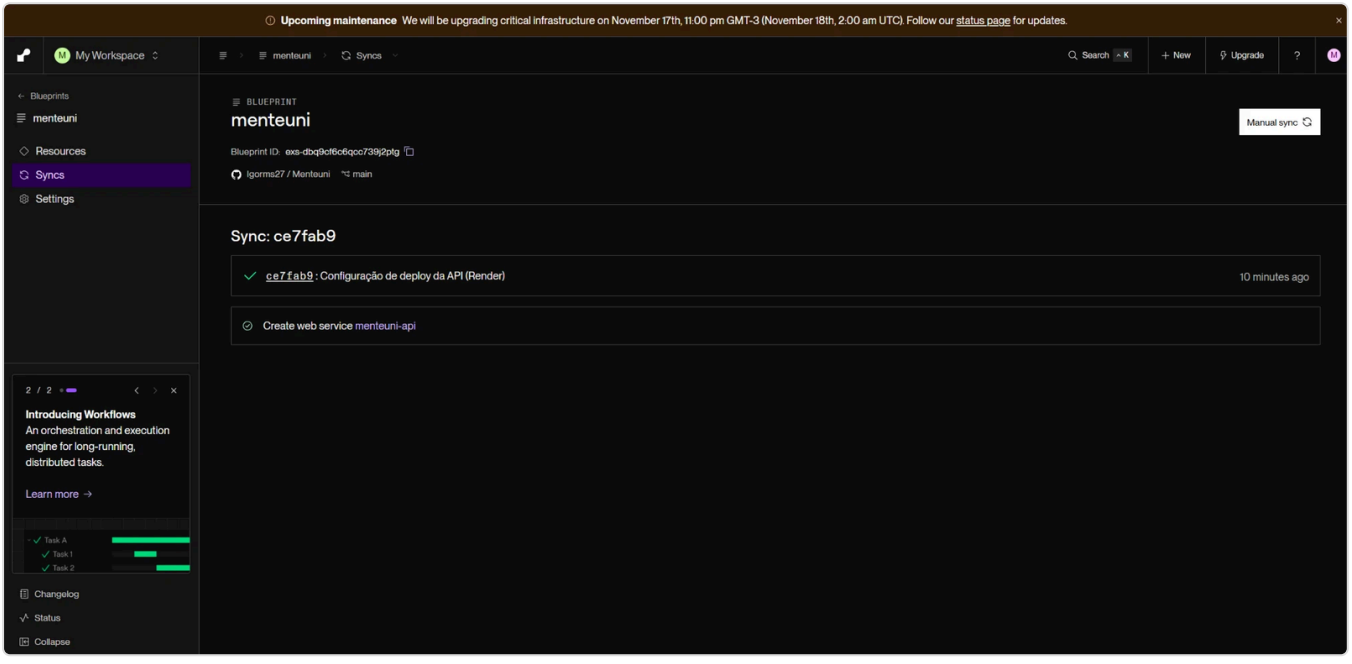
3.3 Banco de dados de produção (Aiven for MySQL)



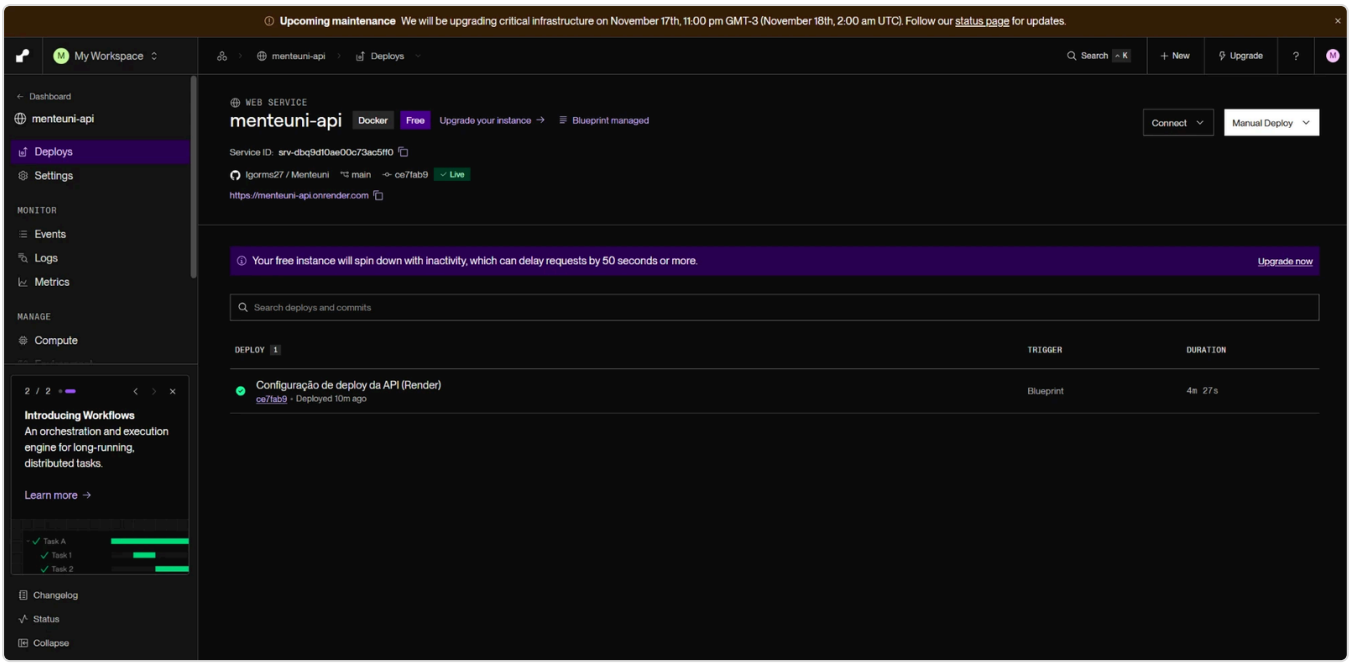
Serviço menteuni-db (MySQL 8.4) em execução no Aiven, com SSL obrigatório. A senha não é exibida.

As tabelas são criadas automaticamente pelo Flyway na primeira inicialização da API (`V1__schema_inicial.sql`), e os questionários e conteúdos iniciais são carregados em `V2__dados_iniciais.sql`.

3.4 API em produção (Render)



Blueprint sincronizado com o repositório GitHub (branch main) e criação do serviço menteuni-api.

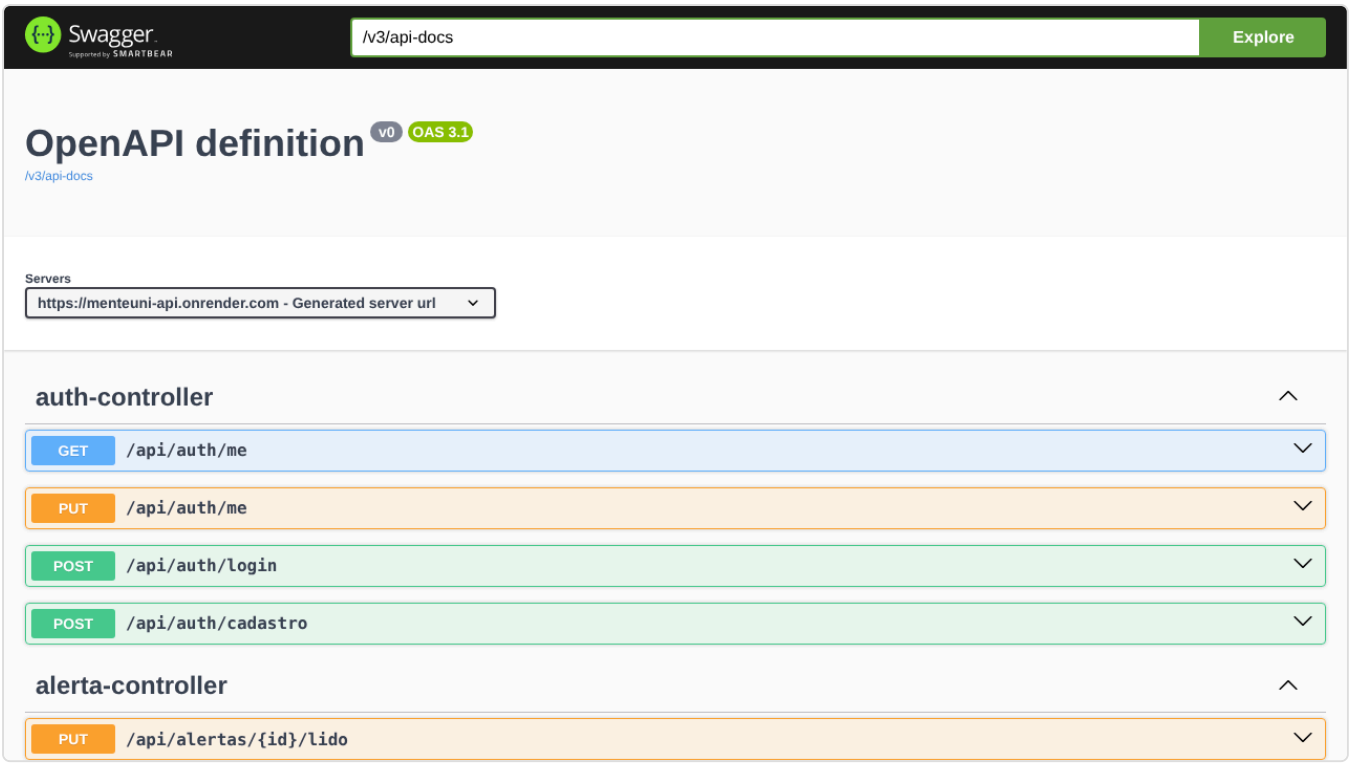


Serviço menteuni-api com status Live no plano gratuito.

Pretty-print

{"aplicacao":"MenteUni API","status":"online","documentacao":"/swagger-ui.html"}

Endereço principal da API respondendo em produção.



Documentação da API (Swagger/OpenAPI) em produção.

Para validar o ambiente, foi feito um teste completo contra a API em produção: cadastro de usuário, login, check-in, listagem de questionários e conteúdos, login com senha incorreta (retorno 401) e verificação de CORS para a origem do app Android (`https://localhost`). **Todos os testes passaram.**

4. Assinatura do app (App Signing)

Foi gerada a chave de assinatura definitiva (`.jks`) com o `keytool` do JDK. O arquivo da chave e as senhas ficam em `keystore.properties`, fora do controle de versão (listados no `.gitignore`), e foram guardados pela equipe.

```
keytool -genkeypair -v -keystore menteuni-release.jks -alias menteuni \
  -keyalg RSA -keysize 2048 -validity 10000 \
  -dname "CN=MenteUni, OU=Projeto Mobile, O=UNICEPLAC, L=Brasilia, ST=DF, C=BR"

Alias name: menteuni
Owner: CN=MenteUni, OU=Projeto Mobile, O=UNICEPLAC, L=Brasilia, ST=DF, C=BR
Valid from: Wed Nov 11 2026 until: Sun Mar 29 2054
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 2048-bit RSA key
```

Configuração no `android/app/build.gradle`:

```
def keystorePropsFile = rootProject.file('keystore.properties')
def keystoreProps = new Properties()
if (keystorePropsFile.exists()) { keystoreProps.load(new FileInputStream(keystorePropsFile)) }

signingConfigs {
    release {
        storeFile rootProject.file(keystoreProps['storeFile'])
        storePassword keystoreProps['storePassword']
        keyAlias keystoreProps['keyAlias']
        keyPassword keystoreProps['keyPassword']
    }
}
```

Verificação da assinatura do APK gerado:

```
$ apksigner verify --print-certs MenteUni-v1.0.apk
Signer #1 certificate DN: CN=MenteUni, OU=Projeto Mobile, O=UNICEPLAC, L=Brasilia, ST=DF, C=BR
Signer #1 certificate SHA-256 digest:
16471a32b03facdbf3625de55aa961eb2ff3fb769ebf414938b21fb1856a40eb
```

5. Otimização e ofuscação (R8/ProGuard)

O build de release ativa o R8, que ofusca o código, remove classes e recursos não usados e reduz o tamanho final do binário.

```
buildTypes {
    release {
        minifyEnabled true          // R8: ofuscação e remoção de código não usado
        shrinkResources true        // remove recursos não utilizados
        proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-
rules.pro'
        signingConfig signingConfigs.release
    }
}
```

Regras adicionadas em `proguard-rules.pro` para preservar as classes do Capacitor, que são acessadas por reflexão a partir do JavaScript:

```
-keep class com.getcapacitor.** { *; }
-keep @com.getcapacitor.annotation.CapacitorPlugin public class * { *; }
-keepclassmembers class * { @com.getcapacitor.PluginMethod public <methods>; }
-keepclassmembers class * { @android.webkit.JavascriptInterface <methods>; }
```

Comandos de build e resultado:

```
npm run build                # build Angular de produção
npx cap sync android         # copia o app web para o projeto Android
./gradlew assembleRelease    # APK assinado
./gradlew bundleRelease      # AAB assinado

app/build/outputs/apk/release/app-release.apk       3,5 MB
app/build/outputs/bundle/release/app-release.aab    3,8 MB
app/build/outputs/mapping/release/mapping.txt       57.519 linhas (mapa da ofuscação R8)
```

6. Identidade visual final

Os ícones adaptativos (camadas de frente e de fundo, Android 8+) e a splash screen foram gerados a partir da logomarca do projeto com a ferramenta `@capacitor/assets`, para todas as densidades de tela (mdpi a xxxhdpi), retrato, paisagem e modo escuro: 74 arquivos no total.

```
plugins: {
  SplashScreen: { launchShowDuration: 1500, launchAutoHide: true,
                  backgroundColor: '#f8f6f1', showSpinner: false, androidScaleType: 'CENTER_CROP'
}
}
```



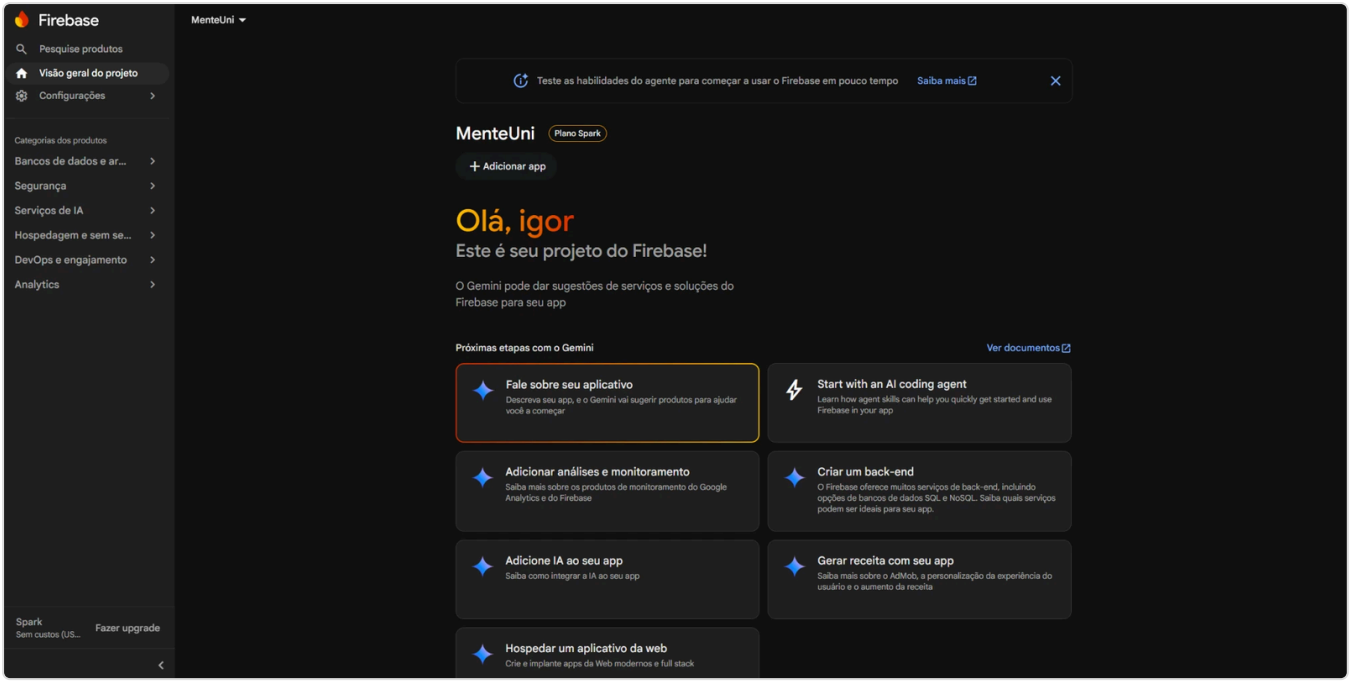
Ícone nas densidades mdpi a xxxhdpi.



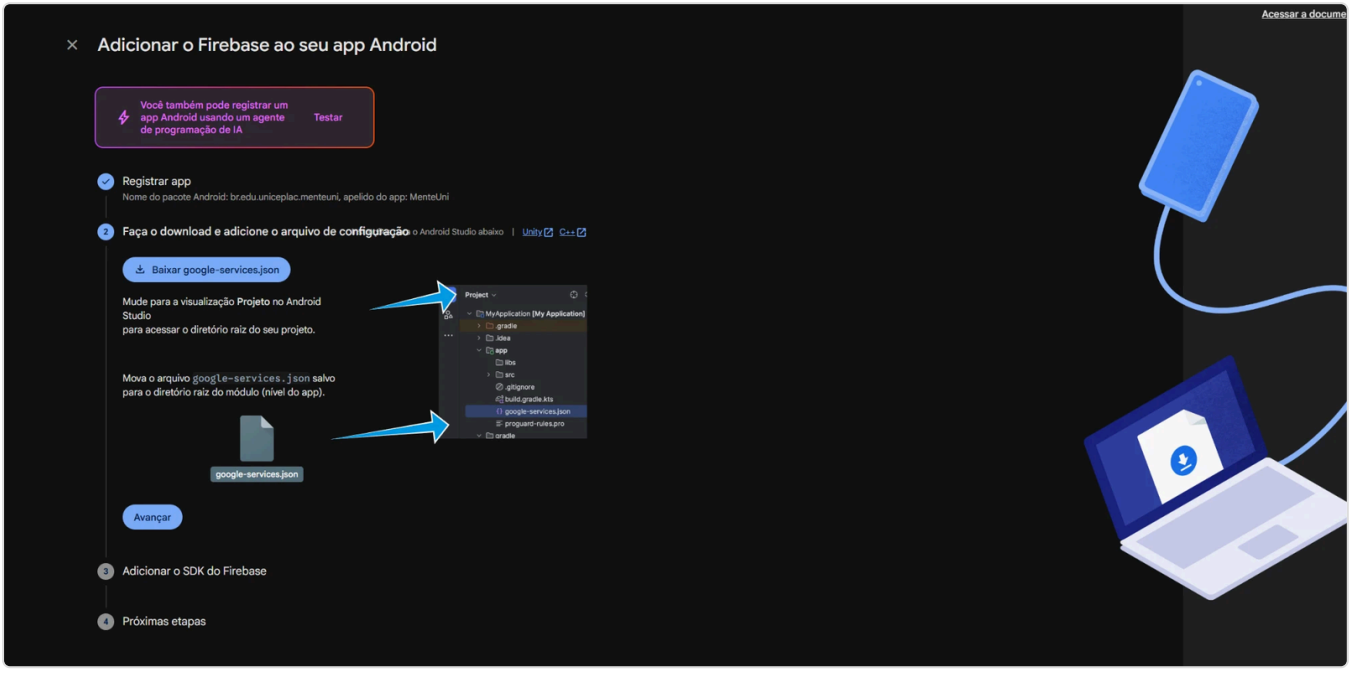
Splash screen (retrato).

7. Distribuição: Firebase App Distribution

7.1 Projeto e app no Firebase

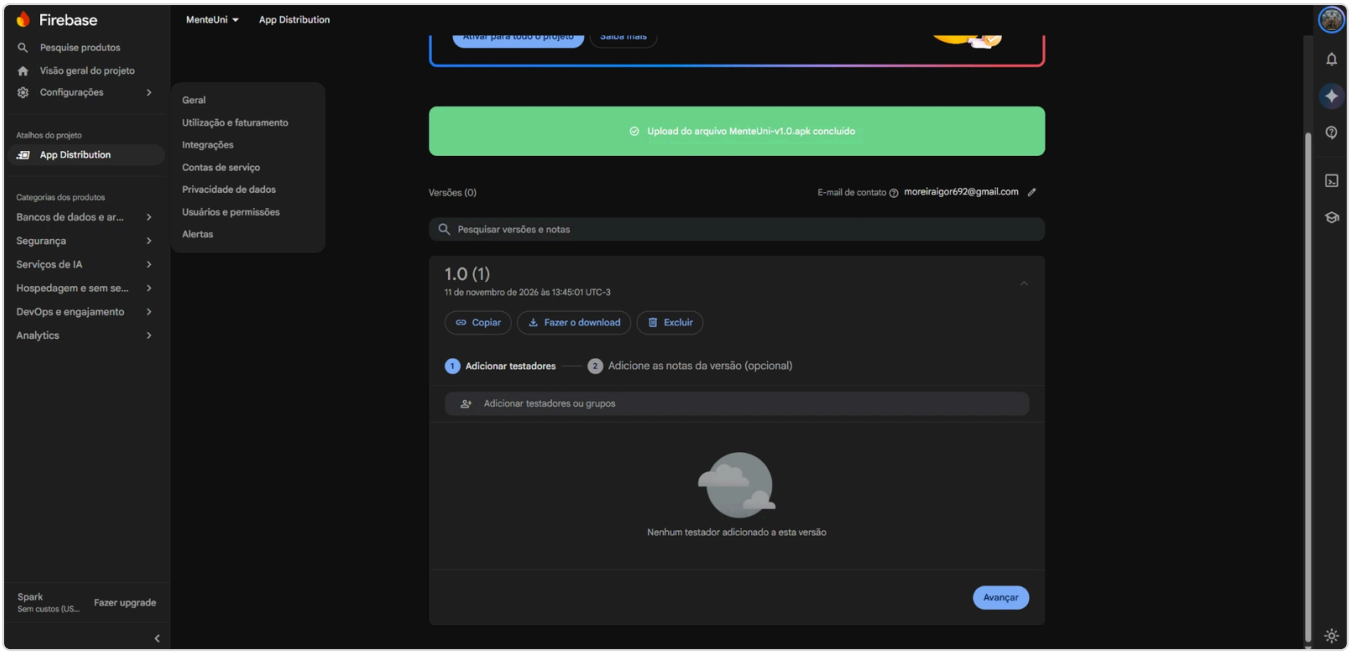


Projeto MenteUni criado no Firebase (plano Spark, sem custos).

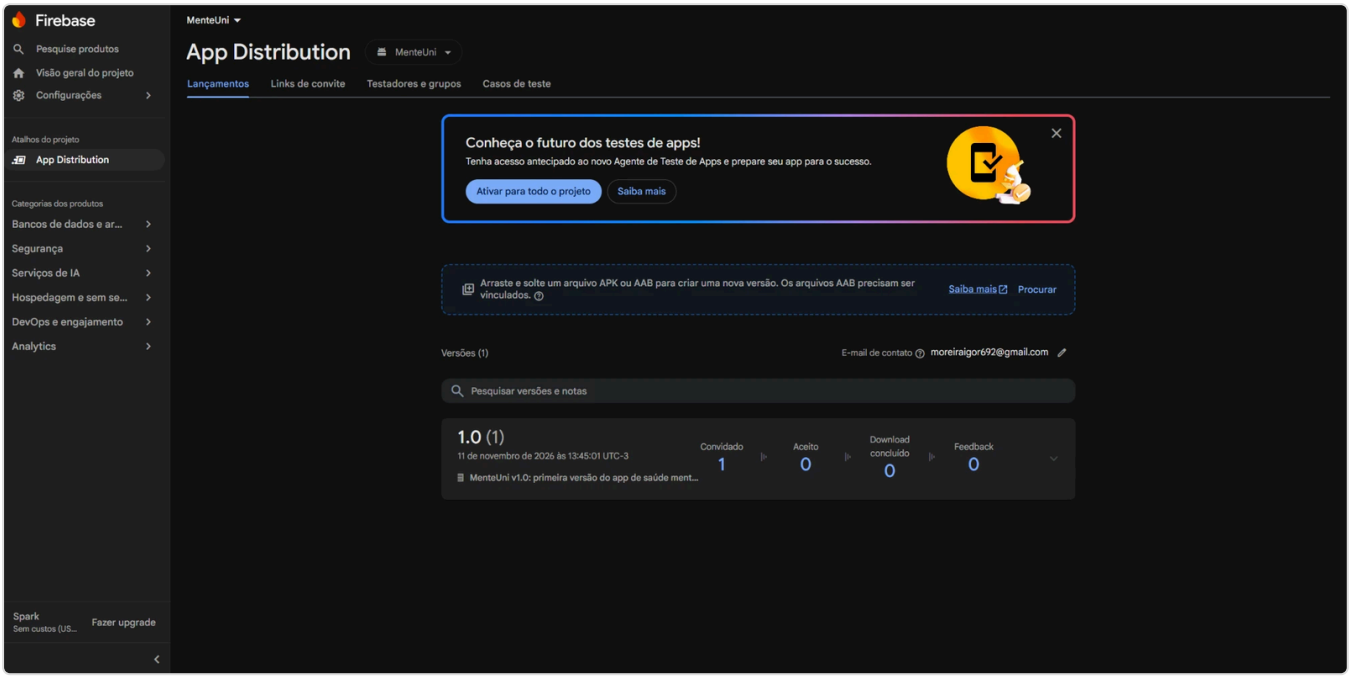


App Android registrado com o pacote br.edu.uniceplac.menteuni.

7.2 Envio da versão

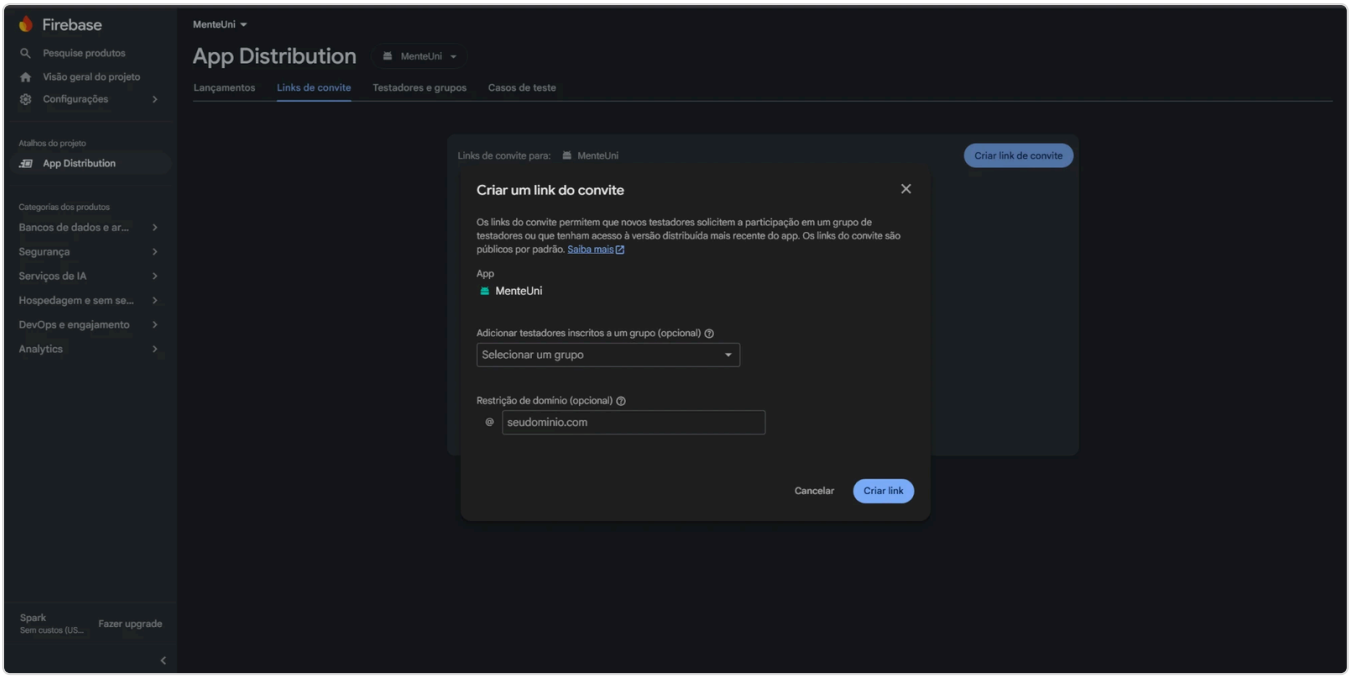


Upload do arquivo MenteUni-v1.0.apk concluído (versão 1.0 (1)).



Versão 1.0 distribuída aos testadores, com as notas da versão.

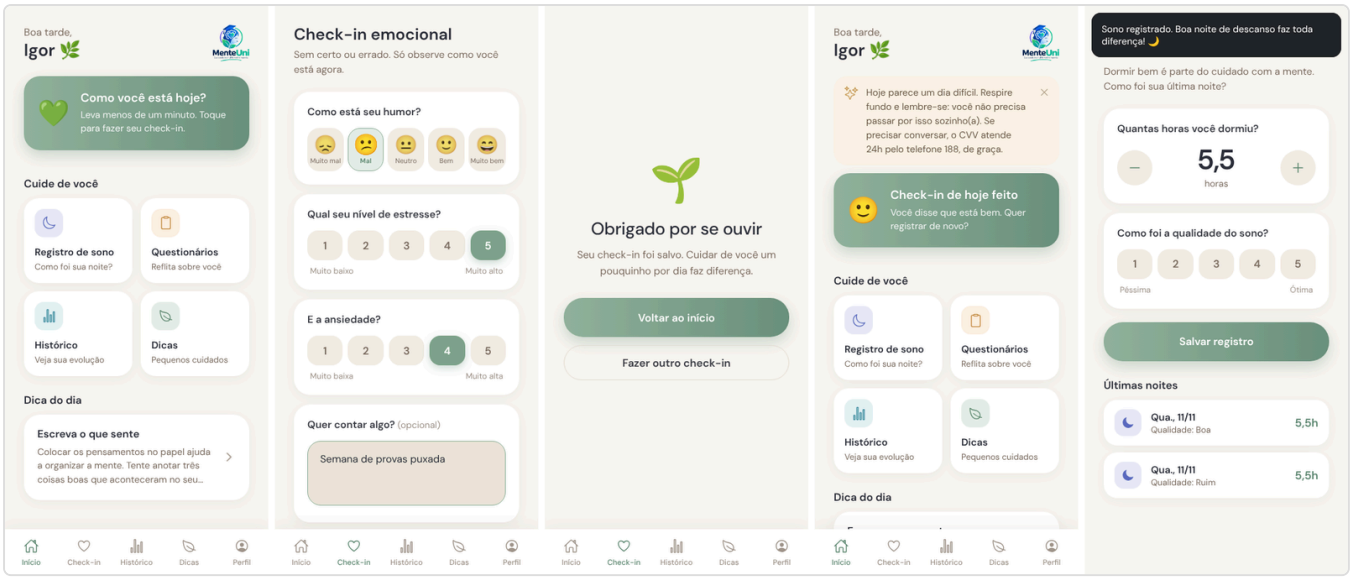
7.3 Link de convite



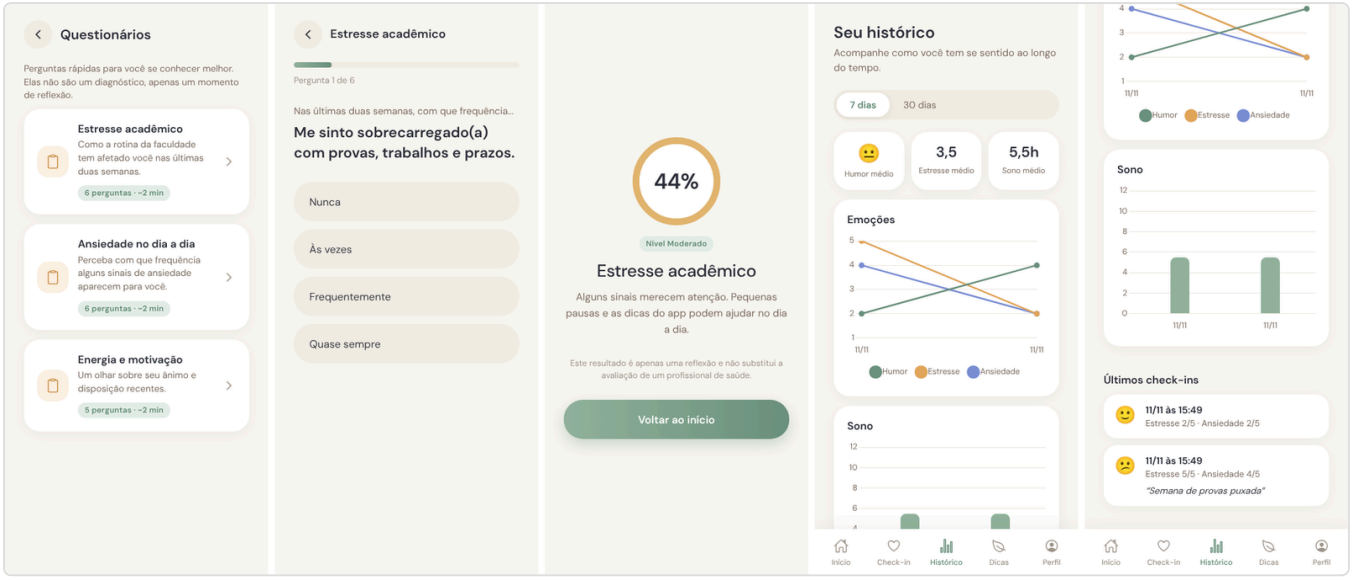
Criação do link de convite público (sem restrição de domínio).

> **Link de convite: https://appdistribution.firebase.dev/i/7d180195a46b49c6**
> Os testadores (colegas de outras turmas, amigos e familiares) recebem o convite por e-mail ou pelo link acima. Pelo celular Android, eles aceitam o convite e instalam o app.

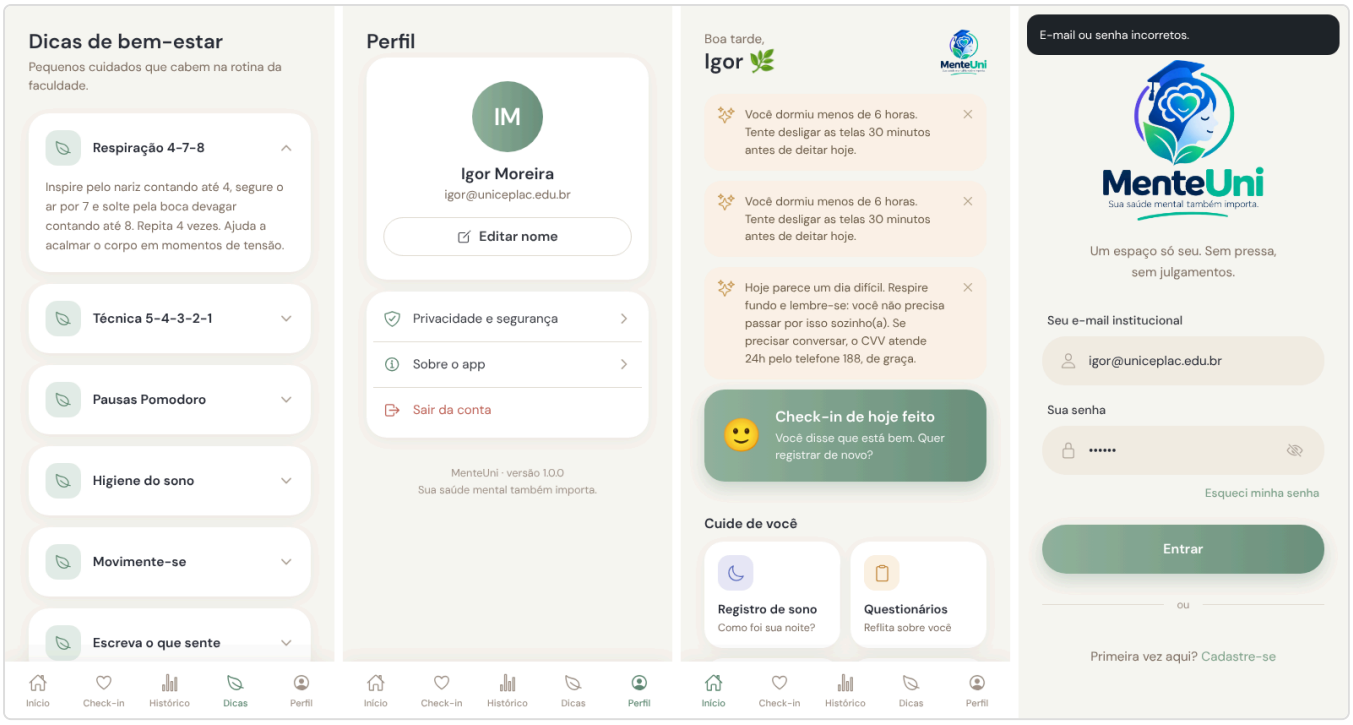
8. Aplicativo em funcionamento



Início, check-in emocional, confirmação do check-in, início com alerta automático e registro de sono.



Questionários, pergunta, resultado e histórico com gráficos de emoções e de sono.



Dicas de bem-estar, perfil, início com alertas e login com mensagem de erro.

9. Considerações

- Os serviços de produção usam planos gratuitos. No Render, a API entra em hibernação após um período sem uso, e o primeiro acesso pode levar até cerca de 1 minuto. O app acorda o servidor ao abrir e avisa o usuário durante a conexão.
- Senhas e chaves (banco, JWT, keystore) ficam somente nas variáveis de ambiente dos serviços e em arquivos fora do repositório.
- O app não substitui acompanhamento profissional e indica o CVV (188) nas telas de dicas e nos alertas.